

## Section 3 / Problem 2 — Food Ordering System using gRPC

---

A Swiggy-style ordering system: one server, any number of customer and restaurant clients, all talking over gRPC. Customers browse menus, place, track and cancel orders; restaurants accept and process them; status changes are pushed to tracking customers in real time via a server-streaming RPC.

---

### Files

| File                               | Purpose                                                              |
|------------------------------------|----------------------------------------------------------------------|
| <code>food_ordering.proto</code>   | Service + message definitions                                        |
| <code>server.py</code>             | Server: restaurants, orders, state machine, streaming                |
| <code>customer.py</code>           | Customer CLI                                                         |
| <code>restaurant.py</code>         | Restaurant CLI                                                       |
| <code>test_system.py</code>        | Automated end-to-end test (concurrency + all error cases)            |
| <code>setup.sh</code>              | One-time: makes a venv, installs gRPC, generates stubs               |
| <code>generate_stubs.sh</code>     | Regenerates the <code>_pb2</code> files from the <code>.proto</code> |
| <code>requirements.txt</code>      | Python dependencies                                                  |
| <code>food_ordering_pb2*.py</code> | <b>Generated</b> by <code>generate_stubs.sh</code> — do not edit     |

---

### Setup (once)

PROF

```
cd "Section 3/Q2_food_ordering"
./setup.sh
```

This creates a `venv/` folder, installs `grpcio` + `grpcio-tools`, and generates `food_ordering_pb2.py` and `food_ordering_pb2_grpc.py`.

Then, **in every new terminal you open**, activate the environment first:

```
source venv/bin/activate
```

If you edit `food_ordering.proto` later, run `./generate_stubs.sh` again.

---

# Running

Open three terminals in this folder (activate the venv in each).

## Terminal 1 — server

```
python3 server.py localhost:50051
```

## Terminal 2 — customer

```
python3 customer.py localhost:50051
```

## Terminal 3 — restaurant (log in as one of the two restaurants)

```
python3 restaurant.py localhost:50051 "Pizza House"
```

Open as many customer/restaurant terminals as you like — e.g. a 4th one as "Burger Point".

**Automated test** (starts its own server; no other terminal needed):

```
python3 test_system.py
```

Should end with **ALL CHECKS PASSED**.

---

## Commands

—  
PROF

**Customer** (numbers 1–6 also work):

|                                                        |                       |
|--------------------------------------------------------|-----------------------|
| restaurants                                            | list restaurants and  |
| menus                                                  |                       |
| order <restaurant> "<item>" <qty> ["<item>" <qty> ...] |                       |
| status <order_id>                                      | one-shot status check |
| track <order_id>                                       | live updates in the   |
| background                                             |                       |
| cancel <order_id>                                      |                       |
| exit                                                   |                       |

**Restaurant** (numbers 1–5 also work):

```
pending                orders for this restaurant not yet finished
accept <order_id>      PLACED      -> ACCEPTED
prepare <order_id>     ACCEPTED    -> PREPARING
ready <order_id>       PREPARING   -> READY
exit
```

---

## Demonstration

Three terminals, one machine. The customer subscribes to the order and then sits idle; the restaurant drives the order from a separate terminal.

### Terminal 1 — server

```
$ python3 server.py localhost:50051
[Server] Food Ordering Server listening on localhost:50051
```

### Terminal 2 — customer

```
$ python3 customer.py localhost:50051
[Client] Connected. Type 'help' for commands.

> restaurants
[Server]
1. Pizza House
   - Margherita Pizza : 250
   - Farmhouse Pizza  : 350
   - Garlic Bread     : 150
2. Burger Point
   - Veg Burger       : 180
   - Cheese Burger    : 220
   - French Fries     : 120

> order Pizza House "Margherita Pizza" 1 "Garlic Bread" 2
[Client] Order placed successfully.
[Client] Order ID: 0101
[Client] Total: 550
[Client] Status: PLACED

> track 0101
[Client] Tracking order 0101...

[Update] Order 0101 : PLACED
[Update] Order 0101 : ACCEPTED
[Update] Order 0101 : PREPARING
[Update] Order 0101 : READY
[Client] Tracking of 0101 finished (order reached a final state).
```

```
> status 0101
[Client] Order 0101 : READY
[Client]   Pizza House | Margherita Pizza x1, Garlic Bread x2 | Total:
550

> order Taco Town "Burrito" 1
[Error] Restaurant 'Taco Town' does not exist. (NOT_FOUND)

> exit
[Client] Bye.
```

The four **[Update]** lines arrived without anything being typed here. The only commands issued were **track 0101**, and then **status 0101** after the order had already finished.

### Terminal 3 — restaurant

```
$ python3 restaurant.py localhost:50051 "Pizza House"
[Restaurant] Logged in as 'Pizza House'. Type 'help' for commands.

> pending
[Restaurant] Order 0101 : PLACED   (Margherita Pizza x1, Garlic Bread x2
| Total: 550)

> accept 0101
[Restaurant] Order 0101 : ACCEPTED

> prepare 0101
[Restaurant] Order 0101 : PREPARING

> ready 0101
[Restaurant] Order 0101 : READY

> prepare 0101
[Error] Invalid order state transition: READY -> PREPARING
(FAILED_PRECONDITION)

> exit
[Restaurant] Bye.
```

### Concurrent clients

Open a second customer and place an order while the first is tracking, and a second restaurant as **"Burger Point"**. Each order gets a unique id and each restaurant sees only its own orders.

`test_system.py` also exercises this automatically: 20 simultaneous `PlaceOrder` calls produce 20 unique ids, and 20 simultaneous `accept` calls on one order produce exactly one success with 19 rejections (`FAILED_PRECONDITION`).

## Exception handling

```
> order Taco Town "Burrito" 1
[Error] Restaurant 'Taco Town' does not exist. (NOT_FOUND)

> order Pizza House "Sushi" 1
[Error] Item 'Sushi' is not available at Pizza House. (NOT_FOUND)

> status 0999
[Error] Order 0999 does not exist. (NOT_FOUND)

> cancel 0101                                (after it was accepted)
[Error] Order 0101 cannot be cancelled: it is already ACCEPTED.
(FAILED_PRECONDITION)

> prepare 0101                                (from restaurant "Burger Point")
[Error] Order 0101 belongs to Pizza House, not Burger Point.
(PERMISSION_DENIED)

> prepare 0101                                (when it is already READY)
[Error] Invalid order state transition: READY -> PREPARING
(FAILED_PRECONDITION)
```

| Situation                                                         | gRPC status code    |
|-------------------------------------------------------------------|---------------------|
| Non-existent restaurant / item / order                            | NOT_FOUND           |
| Empty order or non-positive quantity                              | INVALID_ARGUMENT    |
| Cancel after PLACED, or any illegal transition                    | FAILED_PRECONDITION |
| Restaurant touches another restaurant's order, or tries to cancel | PERMISSION_DENIED   |

Where each required demonstration point appears

| Required                          | Where                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------|
| Listing restaurants               | terminal 2, <code>restaurants</code>                                        |
| Placing an order                  | terminal 2, order <code>0101</code> , total 550                             |
| A restaurant processing the order | terminal 3, <code>accept</code> / <code>prepare</code> / <code>ready</code> |

| Required                                | Where                                                                                                     |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------|
| A customer receiving real-time updates  | terminal 2, the four <code>[Update]</code> lines                                                          |
| Two clients interacting concurrently    | terminals 2 and 3, plus the 20-client tests                                                               |
| Two exceptional cases with status codes | <code>NOT_FOUND</code> in terminal 2, <code>FAILED_PRECONDITION</code> in terminal 3, and four more above |

## Design notes

**State machine.** `TRANSITIONS` in `server.py` is the single source of truth: `PLACED→{ACCEPTED, CANCELLED}`, `ACCEPTED→{PREPARING, PREPARING→{READY}}`; `READY` and `CANCELLED` are terminal. Every change goes through `_apply_transition`, so no code path can skip validation.

**Concurrency.** gRPC serves each call on a thread-pool thread. All shared state is protected by one `threading.Lock`; critical sections are dict operations only, so contention is negligible and one lock removes any deadlock risk. Order ids come from a counter incremented under the lock, guaranteeing uniqueness under concurrent `PlaceOrders`. Racing `accepts` on one order are serialised by the lock — the first wins, the rest see the order already `ACCEPTED` and receive `FAILED_PRECONDITION`.

**Streaming.** `SubscribeToOrderUpdates` is a server-streaming RPC. Each subscriber owns a `queue.Queue`; a status change enqueues one `OrderUpdate` per subscriber (under the lock), and the streaming handler yields from its queue (outside the lock, so a slow client never blocks the server). The first message is the current status so a late subscriber starts in sync; the stream closes itself after `READY/CANCELLED`, and the handler polls `context.is_active()` so a disconnected client is cleaned up.

**Extra RPCs.** The assignment lists five methods; two were added so the required CLI options are possible: `CancelOrder` (customer-only — restaurants cannot cancel via `UpdateOrderStatus`) and `GetPendingOrders` (for the restaurant's "View Pending Orders"). `OrderRequest` carries both the order fields and an `order_id`, because the assignment reuses it as the input of `SubscribeToOrderUpdates`.

## Troubleshooting

- `ModuleNotFoundError: No module named 'grpc'` — you forgot source `venv/bin/activate` in this terminal.
- `No module named 'food_ordering_pb2'` — run `./generate_stubs.sh`.

- **AttributeError: module 'food\_ordering\_pb2' has no attribute ...** — the generated files are stale; run `./generate_stubs.sh` again.
- **failed to connect to all addresses** — the server isn't running, or the client is using a different `host:port` than the server.
- **Running across machines / on the cluster** — start the server with `python3 server.py 0.0.0.0:50051` so it accepts outside connections, and point clients at `<server-hostname>:50051`.